

Contents

heartbeat-demo	1
Contents	1
Requirements	1
Installation	2
Note Visualiser and Test Harness Controls	2
Documentation	2
Memory Map	3
Credits	4
Building from Source	4
Reusing the Player Library in Your Own Project	5

heartbeat-demo

A C/Oscar64 port of the [Heartbeat Soundtracker](#) standalone player, for the Ultimate 64 — plus a hardware-verified demo that plays a real Heartbeat song end-to-end: turbo mode, the 16 MB REU, and all 8 SID chips + 7 Ultimate Audio DMA channels driven from a single tick IRQ.

Status: feature-complete. All 10 phases of the port are done and hardware-verified — full song playback (tempo, all SID + Ultimate Audio channels, vibrato/PWM/filter sweep/portamento modulation, in-pattern track commands) plus a buttons.s-equivalent interactive test harness.

Contents

1. [Requirements](#)
 2. [Installation](#)
 3. [Note Visualiser and Test Harness Controls](#)
 4. [Documentation](#)
 5. [Memory Map](#)
 6. [Credits](#)
 7. [Building from Source](#)
 8. [Reusing the Player Library in Your Own Project](#)
-

Requirements

- **Ultimate 64** (U64 or U64 Elite) with firmware configured as follows:
 - **Turbo Mode** enabled: F2 → Turbo Mode → *U64 Turbo Registers* (the tick IRQ's per-tick modulation work needs turbo headroom — see [ARCHITECTURE.md](#))
 - **REU** set to 16 MB: F2 → C64 settings → REU → *16 MB*
 - **Ultimate Audio** enabled: F2 → C64/Cart settings → *Audio*
 - **Command Interface (UCI)** enabled: F2 → UCI Settings → *Enable*
 - If some SID channels are silent, check F2 → Audio Mixer → Vol UltiSid 1/ Vol UltiSid 2 are not OFF — see [HEARTBEATPLAYERMANUAL.md](#)
- A Heartbeat Soundtracker .reu song file on an SD card or USB drive (see Installation)

Installation

1. Build (see [Building from Source](#)) or download a release ZIP.
2. Extract to the **root** of an SD card or USB drive — the ZIP contains the `idi8b/heartbeat-demo/` folder already, so extracting at the drive root creates the correct layout.
3. Place your Heartbeat Soundtracker `.reu` song file in that same folder, named to match `hb_song_file[]` in `src/main.c` (ships set to "Knight Rider Theme.reu", the reference test song).
4. Insert the SD card / connect the USB drive to your Ultimate 64.
5. In the Ultimate menu, navigate to `idi8b/heartbeat-demo/` and load `heartbeat-demo.prg`.

The demo auto-detects hardware, loads the song (auto-scanning all connected SD/USB drives if it isn't found at the U64's configured home directory first), and starts playback automatically.

Note Visualiser and Test Harness Controls

After hardware detection and song loading, press any key to switch to the note visualiser screen: a horizontal VU-meter bar per active channel (all 7 Ultimate Audio channels, plus 3 rows per populated SID chip), driven live from the player's visualizer event queue (`hb_vis_events[]` — see [HEARTBEAT-PLAYERMANUAL.md](#)). Each channel's bar jumps to peak brightness on a note-on and decays smoothly until the next one, with a green/yellow/red gradient by loudness.

A `buttons.s`-equivalent interactive test harness is active on this same screen:

Key	Action
SPACE	Restart the song from the beginning
RUN/STOP	Silence all sound
A-0	Manually trigger sample FX 1-15 at C-4 on Ultimate Audio channel 6
X	Stop the FX channel (6)
RETURN	Exit to BASIC

If the letter keys don't seem to respond, check that Shift Lock isn't engaged.

Documentation

Document	Covers
HEARTBEATPLAYERMANUAL.md	Public API reference, song file format, track command reference
ARCHITECTURE.md	Internal design: tick/IRQ architecture, data flow, shadow-flush pattern, verification
TURBOCONTROLMANUAL.md	Turbo speed control library
ULTIMATEAUDIOMANUAL.md	Ultimate Audio DMA hardware layer
UCILIBMANUAL.md	Ultimate Command Interface (file/REU access) library

Document	Covers
oscar64manual.md	Oscar64 compiler reference and gotchas encountered during this project
NOTICE.md	Third-party licensing (Heartbeat Soundtracker player source)

A PDF of this README is included in release ZIPs (`README.pdf`, regenerated via `make docs` — see [Building from Source](#)).

Memory Map

Runtime layout for the compiled binary (Oscar64, VIC bank 0, \$01=\$36 — KERNAL + I/O visible, BASIC ROM removed). Exact section sizes shift per build; check `build/heartbeat-demo.map` for the current binary.

Program sections

Range	Contents
\$0801-\$0852	Oscar64 BASIC bootstrap (SYS stub)
\$0400-\$07FF	Text screen RAM (VIC bank 0, 40×25 chars)
\$0A00-\$BFFF	Code + data + BSS + heap + stack (single build region, see <code>#pragma region</code> in <code>src/main.c</code>)

I/O region (\$D000-\$DFFF at \$01=\$36)

Address	Device
\$D000-\$D3FF	VIC-II registers
\$D400-\$D7FF	SID registers (up to 8 chips, addresses from the song's own <code>sid_addresses</code> table)
\$D800-\$DBFF	Color RAM
\$DC00-\$DCFF	CIA 1 (keyboard matrix; Timer A drives the tick IRQ; also a fixed raster IRQ for keyboard scanning)
\$DD00-\$DDFF	CIA 2 (serial bus; VIC bank select)
\$DF00-\$DF1F	REU registers
\$DF20-\$DFFF	Ultimate Audio channels 0-6

Patches applied at startup

Address	Value	Reason
\$0310	\$60 (RTS)	Stub target for KERNAL UDTIM hook redirects
\$A002:\$A003	\$10 \$03	Redirects KERNAL JMP (\$A002) to the RTS stub at \$0310 — prevents the KERNAL

Player runtime state and embedded tables

See [ARCHITECTURE.md](#) for the full breakdown of `hb_songdata/hb_state/hb_sids/hb_ua` and the embedded BPM/frequency tables.

REU (16 MB, \$000000-\$FFFFFF)

Range	Contents
\$000000 +	The loaded .reu song file (header, patterns, samples — see HEARTBEATPLAYERMANUAL.md for the inter

Credits

- **Code:** Xander Mol
 - **Demo framework:** scaffolded from [UltimateDemo2026](#)
 - **Heartbeat Soundtracker player:** © Aleks Eeben / Eight Bit Shed, used and redistributed with permission — see [NOTICE.md](#)
 - **UCI/DOS library:** Scott Hutter & Francesco Sblendorio
 - **Compiler:** [Oscar64](#) by drmortalwombat
-

Building from Source**Prerequisites**

Tool	Purpose	Install
Oscar64	C compiler	see project README
zip	Release archive	<code>sudo apt install zip</code>
wput	FTP deploy	<code>sudo apt install wput</code>
curl	Deploy connectivity check	<code>sudo apt install curl</code>
pandoc + texlive-xetex	PDF docs (optional)	<code>sudo apt install pandoc texlive-xetex</code>

Build

```
make          # compile → build/heartbeat-demo.prg + versioned ZIP in build/
make clean    # remove build artefacts
make docs     # regenerate README.pdf (requires pandoc)
make deploy   # upload PRG + test song to your Ultimate 64 via FTP
```

`make all` (the default target) also regenerates `README.pdf` if `pandoc` is available, so it stays in sync with `README.md` in every release ZIP.

Deployment setup

Copy `.env.example` to `.env` and set your Ultimate 64's IP address:

```
cp .env.example .env
```

```
ULTHOST = 192.168.1.x
```

`.env` is listed in `.gitignore` and will not be committed. `make deploy` checks connectivity first (check-deploy) and prints a clear error if the device is unreachable, instead of failing deep inside `wput`.

Make targets

Target	Does
<code>all</code> (default)	Build <code>.prg</code> , regenerate <code>README.pdf</code> , produce a versioned release ZIP
<code>clean</code>	Remove build/ artefacts
<code>zip</code>	Build the release ZIP alone
<code>docs</code>	Regenerate <code>README.pdf</code> alone
<code>check-deploy</code>	Verify the U64 in <code>.env</code> is reachable (used internally by <code>deploy</code>)
<code>deploy</code>	Upload the <code>.prg</code> and test song to the U64 via FTP

Reusing the Player Library in Your Own Project

`include/hbplayer.h/.c` is a self-contained, reusable library for playing Heartbeat Soundtracker songs in any Oscar64-based Ultimate 64 project. Copy the files, `#include "hbplayer.h"`, and Oscar64's `#pragma compile chain` handles the rest.

Important: building `hbplayer.c` requires the licensed Heartbeat Soundtracker player's binary lookup tables (`reference/heartbeat-player-src/bin/*.bin`), which are **not** included in this repository — see [NOTICE.md](#). You'll need your own Heartbeat Soundtracker license to obtain them.

Files	<code>include/hbplayer.h / include/hbplayer.c</code>
Manual	HEARTBEATPLAYERMANUAL.md
Architecture	ARCHITECTURE.md

```
#include "hbplayer.h"
#include "turbo.h"

turbo_fast(); // required -- see ARCHITECTURE.md

if (hb_load("My Song.reu", 0x000000UL)) {
    hb_detect_ntsc();
    hb_init(0, 1); // start playing from step 0

    // playback now runs in the background via the tick IRQ
    for (;;) {
        // ... your demo code, hb_play_fx() for one-off effects, etc. ...
    }
}
```